

멀티미디어 프로그래밍 과제 2 레포트

22011807 최재현

우선 이미지를 3등분하여 합치는 작업을 했습니다. CutImage 함수에서 원본 이미지를 삼등분 하고 CombineRGB에서 세 이미지를 합쳤습니다. 합치는 과정에서 세개의 이미지에 대해서 밝기를 r, g, b로 지정하여 합쳤더니 컬러 이미지가 나왔습니다.

이후 조금씩 어긋나있는 세개의 이미지를 정렬하는 코드를 작성했습니다. Match 함수는 두 이미지를 인자로 넣으면 두번째 이미지가 첫번째 이미지에 맞기 위한 오프셋 u, x를 반환하는 함수입니다. Match 함수 내에서는 두번째 이미지를 움직이며 getSSD함수를 호출하여 SSD를 구합니다. SSD는 각각의 점들의 차이를 제공하여 평균을 낸 것으로 이미지의 유사도를 얻을 수 있습니다. 이미지를 움직여가며 최소 SSD가 나오는 부분을 찾아 반환했습니다.

Match 함수를 만드는 과정에서 속도와 정확도를 높이기 위해 많은 시행착오가 있었습니다. 최종적으로 사용한 최적화 방법들과 해결 과정을 소개하겠습니다.

첫 번째로 SSD를 구하는 과정에서 모든 점을 검사하는 것이 아니라 일부 점들만 검사하여 속도를 높였습니다. 기존에는 랜덤하게 건너뛰어가며 검사하려고 했지만 최종 결과가 실행할때마다 달라져서 위험하다고 판단하여 정해진 값만큼만 건너뛰어가며 탐색하도록 수정하였습니다. 그리고 더욱 속도를 높이기 위해 SSD 탐색 중에 기존 최소값을 넘어서는 경우 어차피 앞으로 탐색해도 최소값보다 작을 수 없다고 판단하여 반복문을 빠져나가는 방법을 사용했습니다. 하지만 이 과정에서 간과했던 점이 탐색할때마다 탐색하는 픽셀의 수가 다르기 때문에 결과적으로 점들간의 차이 합은 넘어서더라도 평균으로 치면 최소값보다 작을 수 있다는 것입니다. 따라서 이 방법은 결국 사용하지 않았습니다.

두 번째로 매칭하는 이미지를 움직이는 과정에서 모든 좌표를 하나하나 계산하는 것이 아니라 일정 간격씩 점프해가며 최소값을 찾고 그 주위에서 더 자세히 탐색하는 방법을 사용했습니다. 우선 상수 area_jump를 선언하여 첫번째 탐색에서는 이 값만큼 점프 해가며 격자 모양으로 최소값을 찾습니다. 그리고 최소값을 찾은 주위에서 1픽셀씩 움직여가며 정밀 탐색을 합니다. 이 때는 area_jump 크기의 정사각형 만큼만 탐색을 합니다. 이 방법을 사용하여 정확도는 조금 낮아졌을지라도 속도적인 측면에서 20배 이상의 효과를 보였습니다. 상수들을 조금씩 조절해가며 최적의 값을 찾았습니다.

마지막으로 아직 정확도가 낮은 문제가 있었습니다. 제가 판단하기로는 옆의 프레임 때문에 정확도가 낮아진다고 생각했습니다. 따라서 제가 선택한 방법은 SSD에 최대값을 두는 것입니다. 예를 들어 각 점의 차이의 제곱을 더할 때 최대 1000까지만 더한다고 하면 아무리 격자부분에서 픽셀간의 차이가 많이 나도 더했을 때 사소할 것이라고 생각했습니

다. 하지만 이 방법이 결국 실패했던 것은 프레임이 아닌 중요한 점들의 유사성마저 무시해버리는 경향을 보였습니다. 예를 들어 R과 B채널 이미지가 아무리 유사하다고 할지라도 파란 하늘 같은 경우는 밝기 차이가 어느정도 날 수밖에 없습니다. 하지만 이 방법만 사용했을 때 일정 차이 이상 벌어지는 경우 그 차이를 무시해버리게 돼서 부정확한 결과를 가져왔던 것입니다. 결과적으로는 SSD를 구할 때 프레임 부분의 지정 픽셀만큼 무시하는 방법으로 해결했습니다. 이 방법을 사용했을 때 정확성은 높았지만 프레임이 있는 특수성을 고려한 프로그램이라는 점에서 확장성 측면에서 바람직하지 않은 것 같아 아쉽습니다.

최종적으로 세 이미지를 합치는 함수에서 구한 오프셋 만큼 이동시켜 합치는 방법으로 정확한 컬러 이미지를 얻었습니다. 그리고 실행 시간을 구하는 것과 입력 받는 부분을 추가하여 완성했습니다.